

CSC111 Winter 2026 Project 1

Alice Nguyen, Syaikha Faiza Arifin, Tamima Wadageri

February 13, 2026

Running the game

The game can be run by executing `adventure.py`. No additional modules are required beyond standard Python libraries. The game runs in the terminal. Players input commands directly when prompted.

Game Map

The game takes place in a small version of University of Toronto environment connected as follows (Location represent by ID):

```
6  1  2
5  4  3
```

Starting location is: Dorm (6)

Game solution

List of commands:

1. Go South
2. Take Lucky Mug
3. Go East
4. Take USB Drive
5. Go North
6. Take Laptop
7. Go East
8. Take Laptop Charger
9. Go West
10. Go West
11. Drop Laptop
12. Drop USB Drive
13. Drop Laptop Charger
14. Drop Lucky Mug

Lose condition(s)

Description of how to lose the game: Players lose if players run out of moves with max 30 moves.

List of commands:

1. Go South
2. Go North

Keep doing that for 30 times until you run out of moves.

Which parts of your code are involved in this functionality:

```
if not self.ongoing and self.moves_used >= self.max_moves:  
    return f"You ran out of moves (moves used: {self.moves_used}). It's 1pm | you lose."
```

Inventory

1. All location IDs that involve items in the game:

(a) Location 1:

- Name: Bahen
- Item: Laptop
- ID: 1

(b) Location 2:

- Name: Robarts Library
- Item: Laptop Charger
- ID: 2

(c) Location 3:

- Name: Old Vic
- Item: -
- ID: 3

(d) Location 4:

- Name: Myhal
- Item: USB Drive
- ID: 4

(e) Location 5:

- Name: Exam Centre
- Item: Lucky Mug
- ID: 5

(f) Location 6:

- Name: Dorm
- Item: -
- ID: 6

2. Item data:

- (a) For Item 1:
 - Item name: Laptop
 - Item start location ID: 1
 - Item target location ID: 6
 - (b) For Item 2:
 - Item name: USB Drive
 - Item start location ID: 4
 - Item target location ID: 6
 - (c) For Item 3:
 - Item name: Lucky Mug
 - Item start location ID: 5
 - Item target location ID: 6
 - (d) For Item 4:
 - Item name: Laptop Charger
 - Item start location ID: 2
 - Item target location ID: 6
3. Exact command(s) that should be used to pick up an item (choose any one or more items for this example), and the command(s) used to use/drop the item (can copy the list you assigned to `inventory_demo` in the `simulation.py` file)
 - Command to pick up item : Take
 - Command to Use/Drop item : Drop
 4. Which parts of your code (file, class, function/method) are involved in handling the `inventory` command:
 - function method : `def take`
 - function method : `def drop`

Score

1. Briefly describe the way players can earn score in your game. Include the first location in which they can increase their score, and the exact list of command(s) leading up to the score increase:
The player could get score collecting all the item and return that to the dorm.
2. Copy the list you assigned to `scores_demo` in the `simulation.py` file into this section of the report:

```
scores_demo = [
    "go south",
    "take lucky mug",
    "score",
    "go north",
    "drop lucky mug",
    "score"
]

expected_log = [6, 5, 5, 6, 6]
```

```

sim = AdventureGameSimulation('game_data.json', 6, scores_demo)
print("---- CHECKING ----")
print("EXPECTED:", expected_log)
print("ACTUAL   :", sim.get_id_log())

assert expected_log == sim.get_id_log()

```

3. Which parts of your code (file, class, function/method) are involved in handling the **score** functionality:

- function method : `def _find_item`
- function method : `def drop`

Enhancements

1. Describe your enhancement #1 here

- Brief description of what the enhancement is (if it's a puzzle, also describe what steps the player must take to solve it):

Enhancement 1 is an optional battle system called Evolution Arena. It extends the traditional Rock-Paper-Scissors mechanic by introducing power levels and an energy management system. There are four types of moves: rock, paper, scissors, and shadow.

Normal dominance rules:

- Shadow defeats any move with power level 1, regardless of type.

Each move has a power level from 1 to 3. If no number is specified, the power defaults to 1.

Examples:

```

rock
paper 2
scissors 3
shadow 1

```

Energy system:

- Both players start with 3 energy.
- Power 1 costs 0 energy.
- Power 2 costs 1 energy.
- Power 3 costs 2 energy.
- Shadow adds +2 additional energy cost.

If a player attempts to use a move that exceeds available energy, the system automatically replaces the move with rock 1.

Round resolution:

- (a) Check dominance using standard RPS rules.
- (b) If neither move dominates, compare power levels.
- (c) If both type and power are identical, the round is a draw.

Scoring:

- Winner earns 1 point.
- First player to reach 5 points wins the Arena.

Energy regeneration after each round:

- Winner gains +1 energy.
- Loser gains +2 energy.
- Draw: both gain +1 energy.

- Complexity level (choose from low/medium/high): Medium
- Reasons you believe this is the complexity level (e.g., mention implementation details, how much code did you have to add/change from the baseline, what challenges did you face, etc.)
 - Required adding a structured mini-game within the main adventure game.
 - Implemented move parsing that supports optional power levels.
 - Added an energy tracking mechanism for both players.
 - Implemented conditional logic for dominance, power comparison, and draw cases.
 - Added a separate scoring system and win condition (first to 5 points).

Although the enhancement introduces new game mechanics and additional state tracking, it builds directly on simple conditional logic and does not require complex data structures or advanced algorithms. Therefore, it is classified as medium complexity rather than high.

- Name the parts of the code which are involved in this enhancement

Class ArenaPlayer :

- def arena_energy_cost
- def arena_beats
- def arena_resolve_round
- def arena_parse_move
- def arena_enforce_energy
- def arena_ai_choose
- def arena_print_rules
- def arena_prompt_move
- def arena_apply_regen
- def play_evolution_arena

- Copy the list you assigned to `enhancements_demo` in the `simulation.py` file into this section of the report:

```
# ENHANCEMENT 1
enhancement1_demo = [
    "go east",
    "go south",
    "go west",
    "go north",
    "take laptop",
    "quit",
    "inventory"
```

```

]

expected_log = [1, 2, 3, 4, 1]

sim = AdventureGameSimulation('game_data.json', 1, enhancement1_demo)
print("---- CHECKING ----")
print("EXPECTED:", expected_log)
print("ACTUAL  :", sim.get_id_log())

assert expected_log == sim.get_id_log()

# ENHANCEMENT 2
enhancement2_demo = [
    "go east",
    "go south",
    "take laptop charger",
    "undo",
    "undo"
]

expected_log = [1, 2, 3, 2, 1]

sim = AdventureGameSimulation('game_data.json', 1, enhancement2_demo)
print("---- CHECKING ----")
print("EXPECTED:", expected_log)
print("ACTUAL  :", sim.get_id_log())

assert expected_log == sim.get_id_log()

# ENHANCEMENT 3
enhancement3_demo = [
    "go east",
    "take laptop charger",
    "restart",
    "go west"
]

expected_log = [1, 2, 2, 1, 6]

sim = AdventureGameSimulation('game_data.json', 1, enhancement3_demo)
print("---- CHECKING ----")
print("EXPECTED:", expected_log)
print("ACTUAL  :", sim.get_id_log())

assert expected_log == sim.get_id_log()

```